



Lab 7

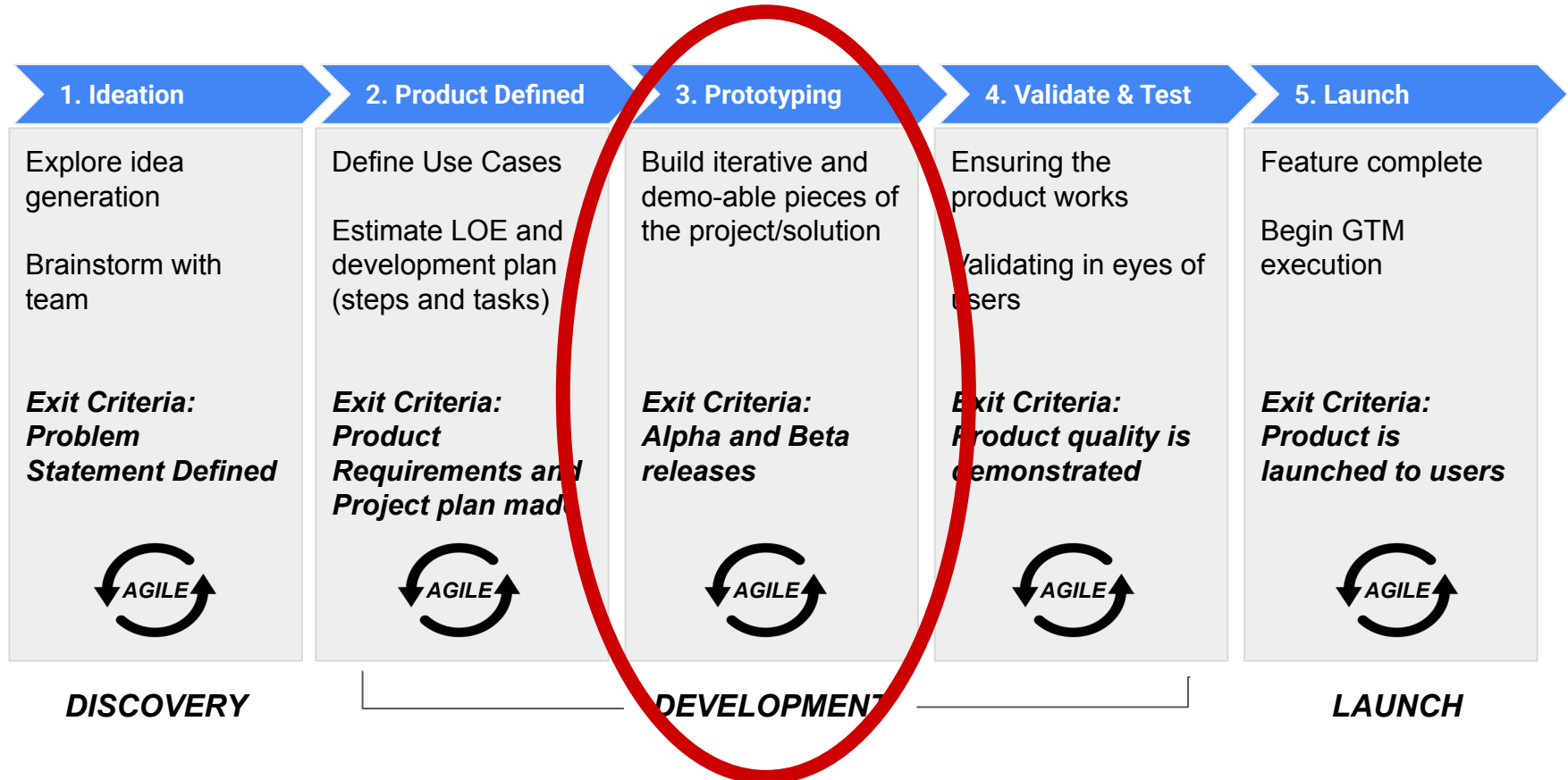
Project Design, User Experience, and Sprint Progress

Agenda

- Project design: where we are now
- UI / UX basics
- User stories and user flows
- Architecture / system design
- Sprint grading and expectations
- TA support
- Upcoming work



Product Definition Phase



Functionality First

- Do not spend too much time making a fancy UI this semester.
- Build enough UI to support basic user stories and demonstrate functionality.
- Spend most development time on core functionality: backend, data, algorithms, integration, and testing.
- Use UI work to clarify user flow and requirements — not as a substitute for working features.

Optional resource: Material Design — use as a reference, not a requirement

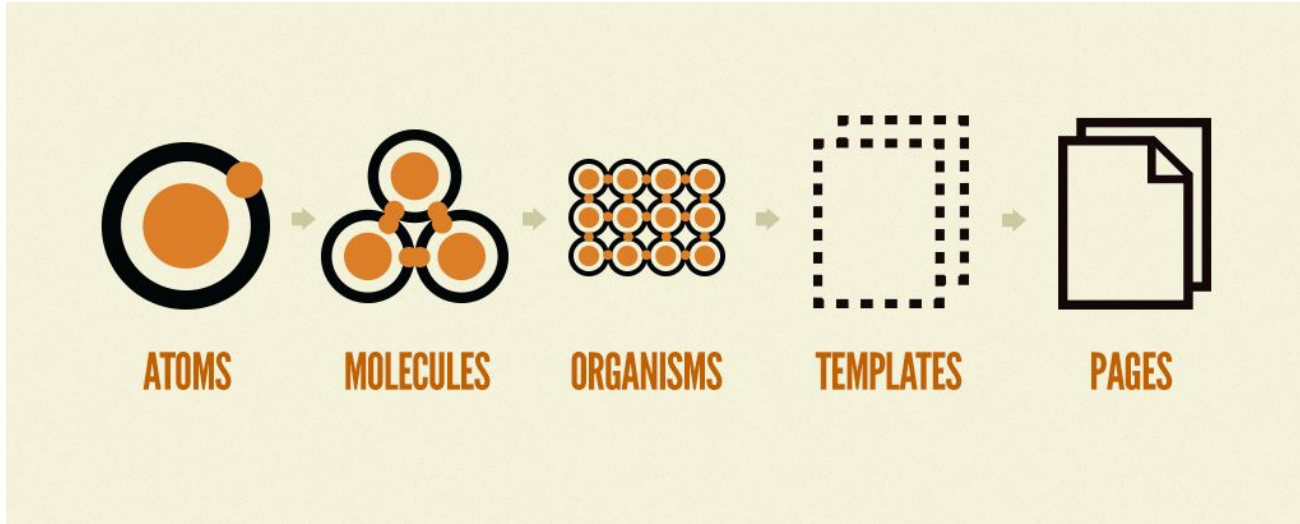
UI Design System

- **What**
 - Set of standards to manage design at scale by creating a shared language using reusable components and patterns
- **Why**
 - Reduces redundancy on Design and Engineering in order to build quicker
 - Creates unified language across cross-functional teams
 - Increases visual consistency across different pages and channels
 - Learning tool for new Designers
- **Atomic Design Theory** has been created to support Design System creation
 - Established by Brad Frost

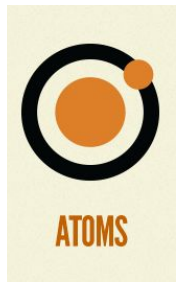
UI Design System

- What's included
- Style guide
- Typography, colors, logos, etc
- Component Library
- Reusable UI elements
- Pattern Library
- Similar to Component but more high-level collections of components
- Design System Team
- Team establishing and maintaining the Design System
- Made of Product Designers, Visual Designers, and Engineers
- For Senior Design: reuse components; do not spend substantial time building a custom design system.

Atomic Design



Atomic Design - Atoms

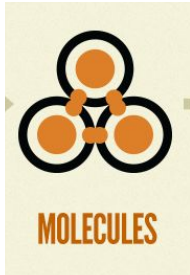


Atoms: Basic HTML Elements

Examples: form labels, inputs, buttons, and others that can't be broken down any further without ceasing to be functional.

SEARCH THE SITE	LABEL
ENTER KEYWORD	INPUT
SEARCH	BUTTON

Atomic Design - Molecules



Molecules: Combination of Atoms

Examples: form label, search input, and button can join together to create a search form molecule

SEARCH THE SITE

ENTER KEYWORD

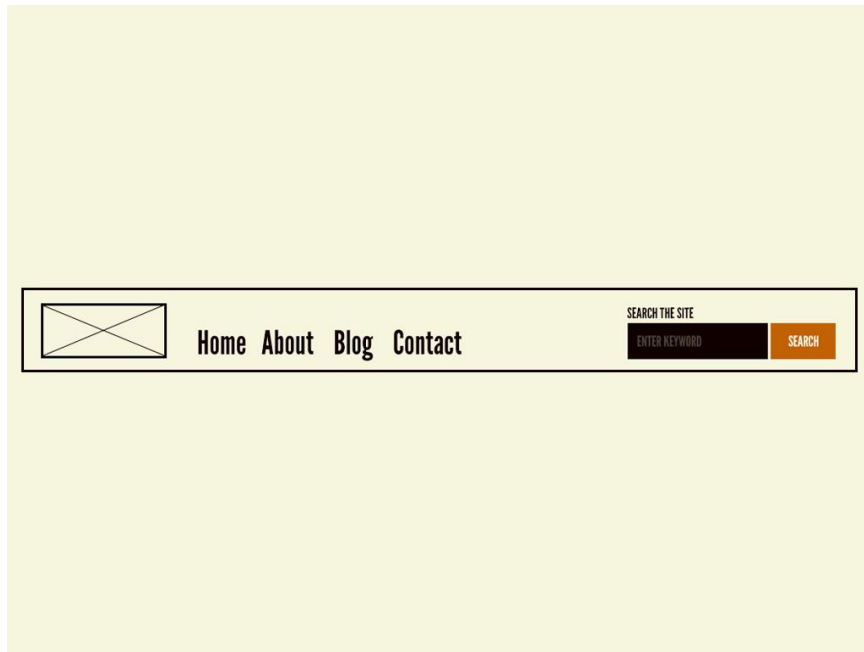
SEARCH

Atomic Design - Organisms



Organisms: complex UI components composed of groups of molecules and/or atoms and/or other organisms

Examples: header navigation, footer navigation, menu panels

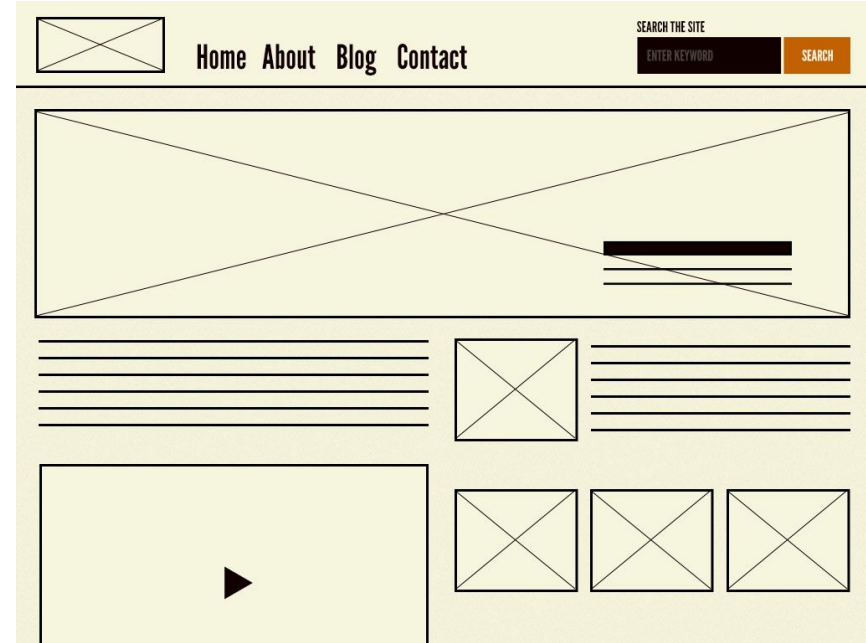


Atomic Design - Templates



Templates: page-level objects that place components into a layout and articulate the design's underlying content structure

Examples: Header combined with visual content with descriptions and action buttons - all laid out in one repeatable space

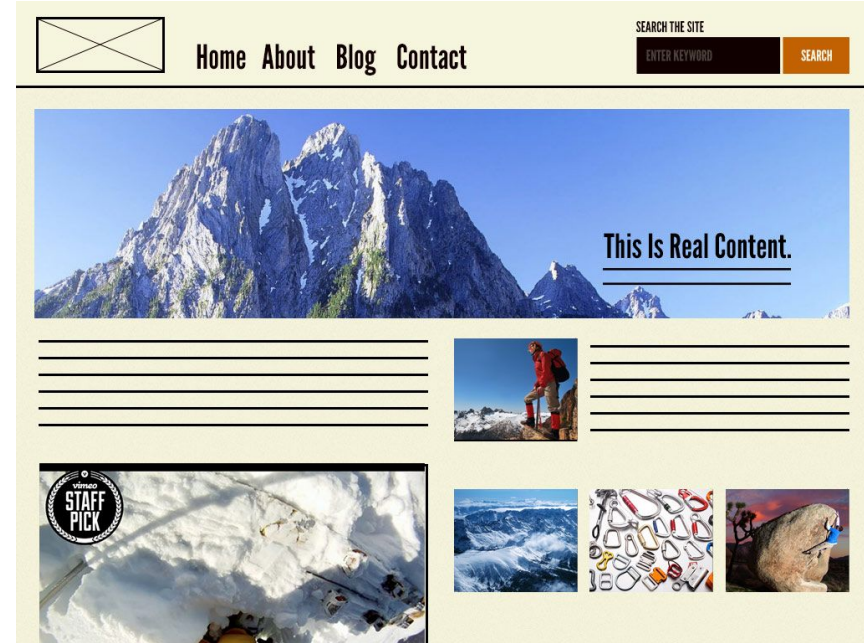


Atomic Design - Pages



Pages: specific instances of templates that show what a UI looks like with real representative content in place

Examples: Home page, Contact Us page, etc



Example Design Systems / Tools

- [Apple Human Interface Guidelines](#)
- [Material Design](#)
- [IBM Design Language](#)
- [Figma](#)
- [Canva — available through GW IT](#)
- [Adobe Creative Cloud / Adobe Express — available through GW IT](#)
- [GW Tools & Services](#)

Use these for basic mockups, flows, and reusable components — not for spending substantial time on visual polish.

UX Design and User Flows

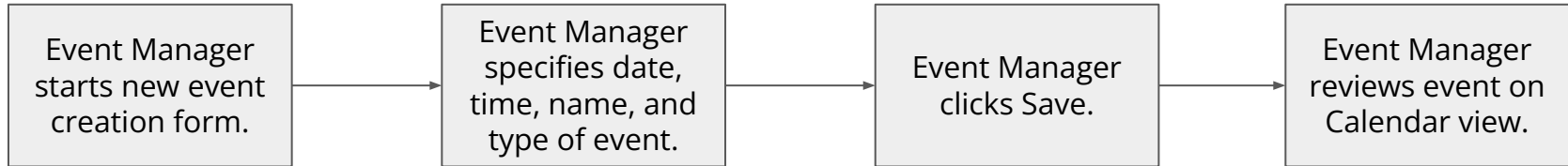
- Built off of User Stories / Use Cases
- Creates high-level app flow

You'll need this for Writing 3

UX Design

- Built off of User Stories / Use Cases

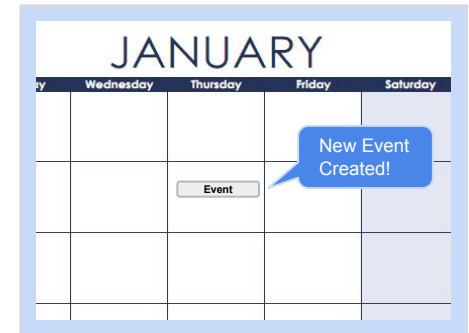
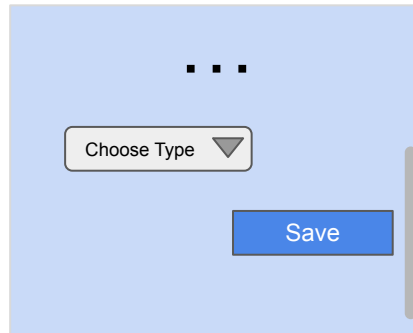
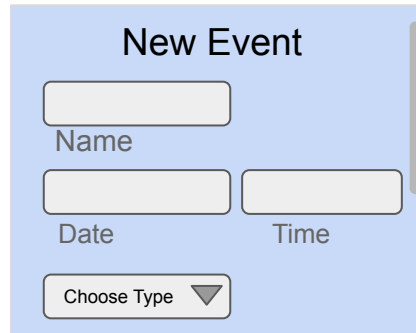
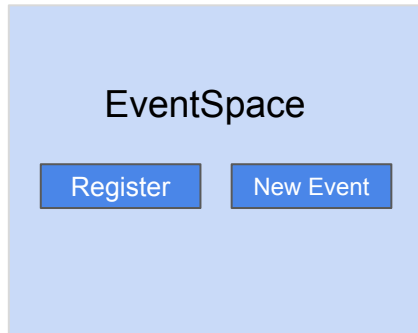
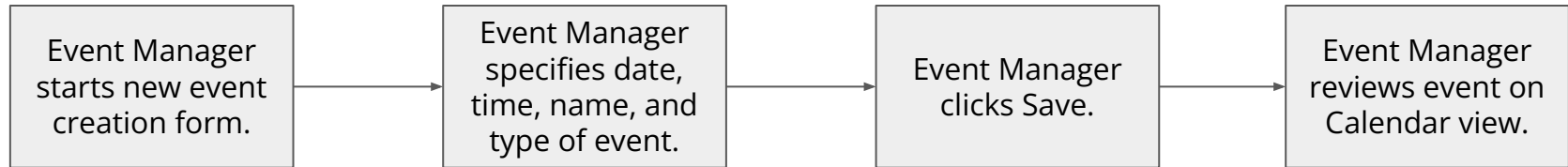
As a Event Manager, I would like to schedule events so that people can see which events are available to register for.



UX Design

- Built off of User Stories / Use Cases

As a Event Manager, I would like to schedule events so that people can see which events are available to register for.



Atomic Design Components

For Senior Design, this level of UI is enough if it supports the required functionality.



New Event

Name

Date

Time

Choose Type

Save

Date

YYYY-MM-DD

December 2021						
M	T	W	T	F	S	S
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Time

8:15 PM

✓ ×

Choose Type

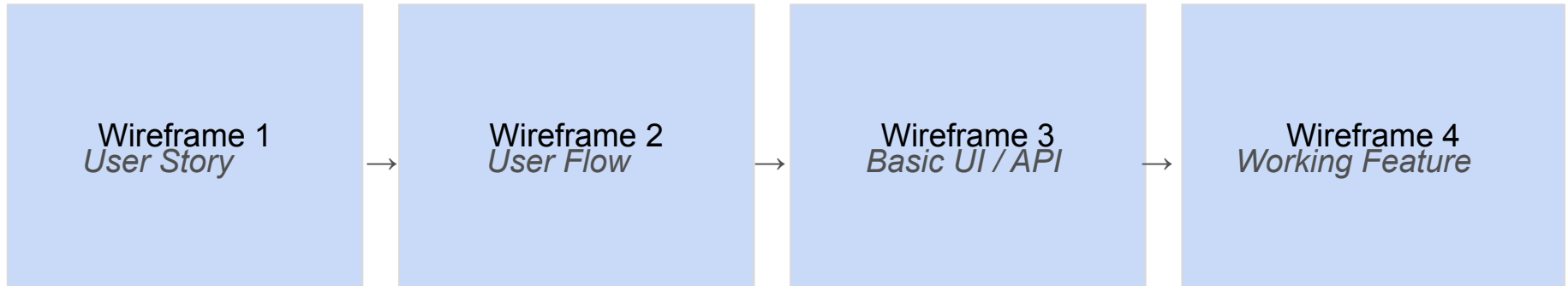
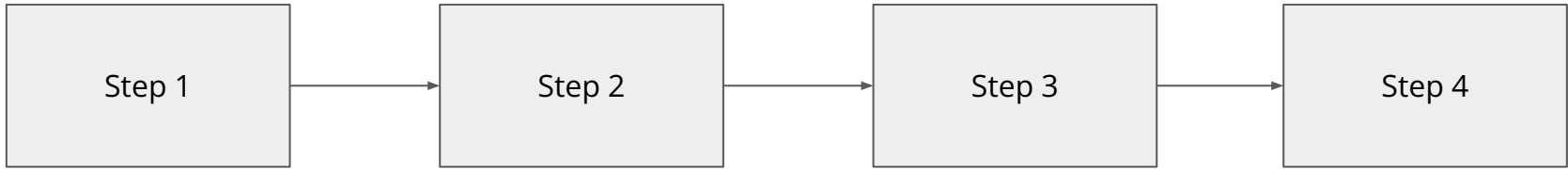
Consult

Check-in

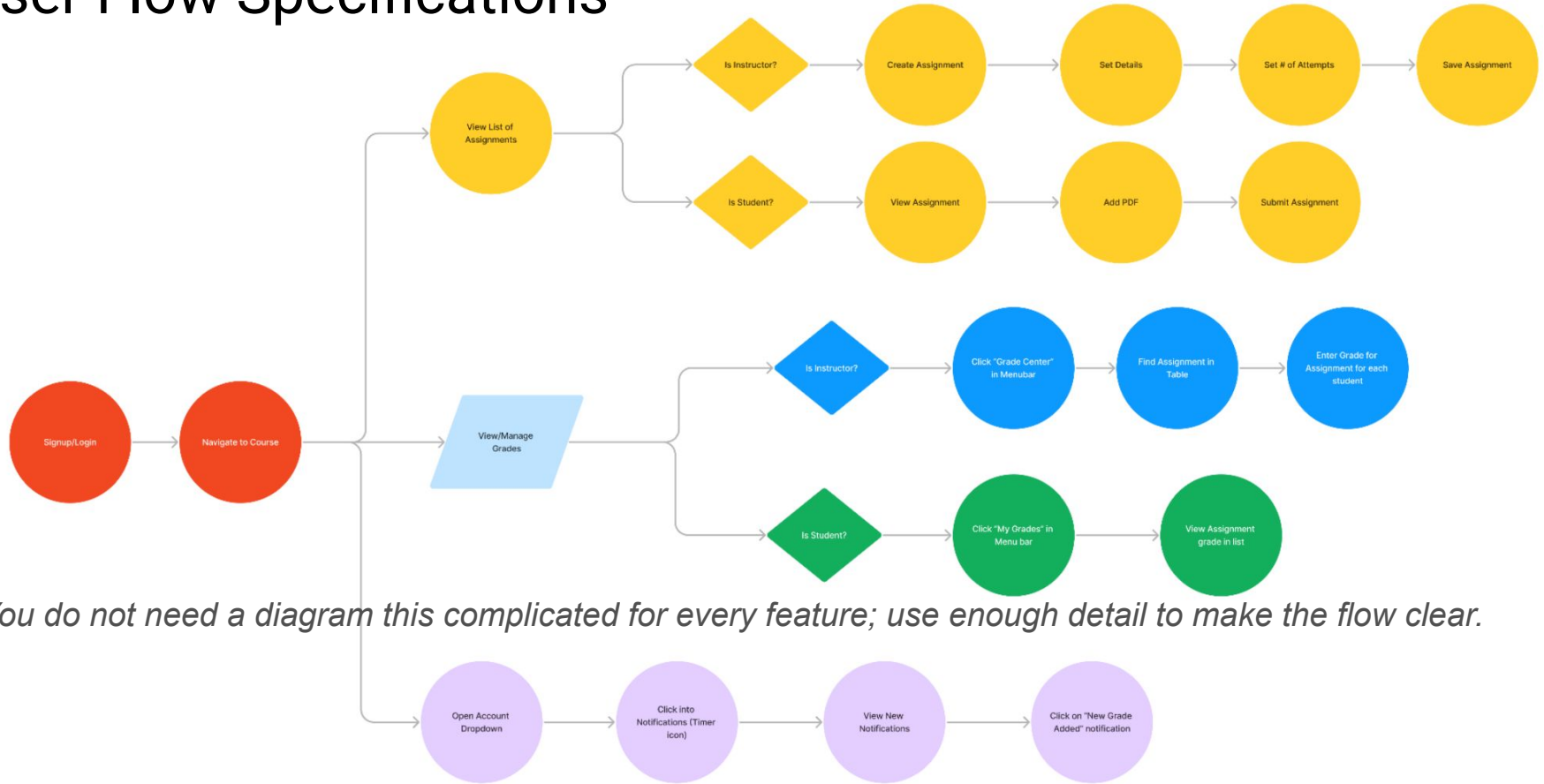
Planning

From User Story to Implementation

As a [persona], I would like to [action] so that [benefit].



User Flow Specifications



You do not need a diagram this complicated for every feature; use enough detail to make the flow clear.

From User Story to Backend / API

UI action → API call → backend processing → stored/retrieved data → result shown to user

As a Event Manager, I would like to schedule events so that people can see which events are available to register for.

Event Manager starts new event creation form.

Event Manager specifies date, time, name, and type of event.

Event Manager clicks Save.

Event Manager reviews event on Calendar view.

POST http://eventSpace.api.com/new_event

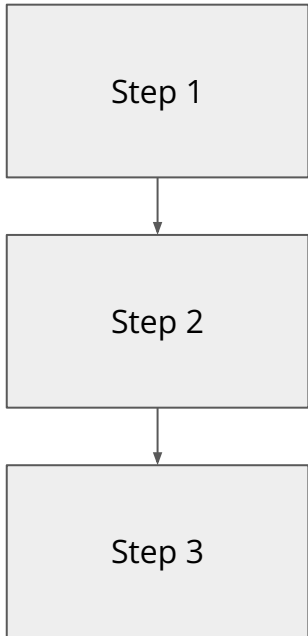
```
{
  "name": "Smith Engagement Party Planning",
  "type": "Planning",
  "start": {
    "date": "2015-05-28",
    "time": "09:00:00-07:00"
  },
  "end": {
    "date": "2015-05-28",
    "time": "17:00:00-07:00"
  }
}
```

GET http://eventSpace.api.com/calendar_events

UX Design for APIs

Optional resource: [GitHub REST API documentation](#)

As a [persona], I would like to [action] so that [benefit].



API Documentation

URL (internal/external)

/name_of_api_call

Request Type

POST, GET, PUT, DELETE

Header Parameters

Authentication, if applicable

Request Body Schema

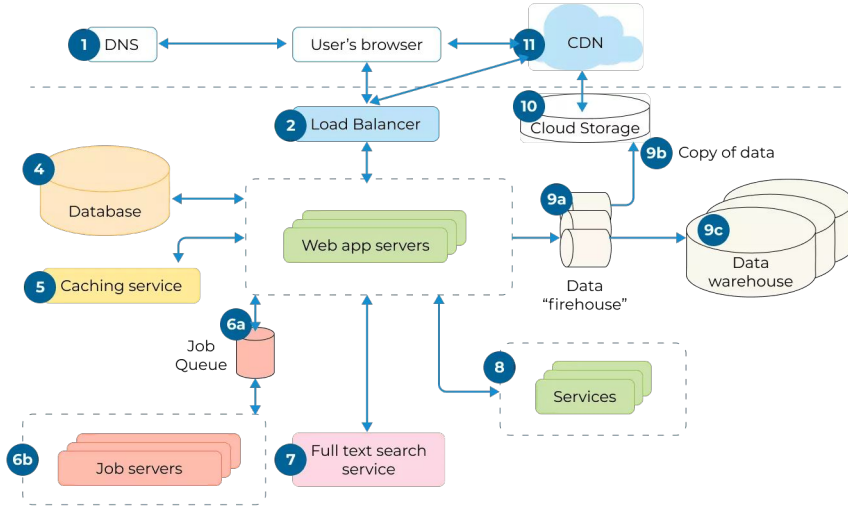
Details passed into call

Architecture/System Design

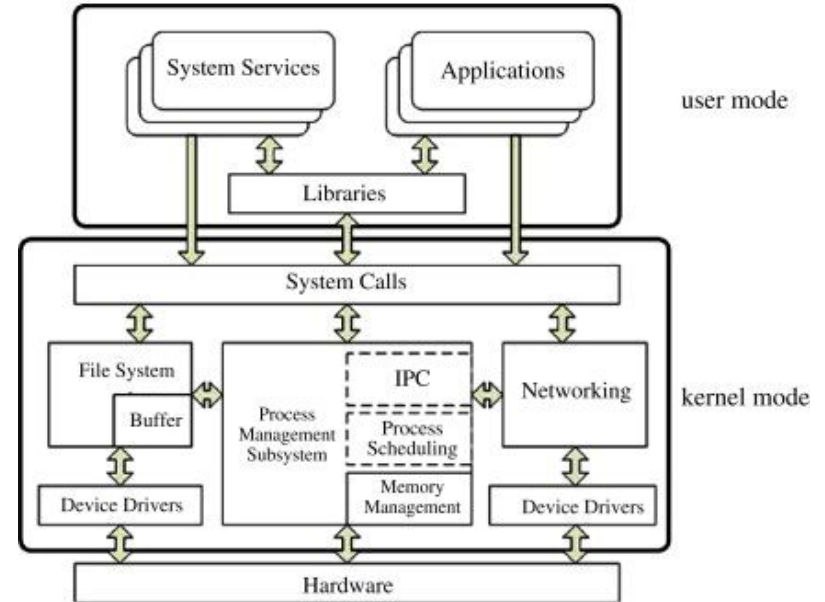
- Diagrams
- Internal/External APIs and Frameworks
- Main Algorithms of System
- Use architecture to explain how the working pieces fit together.

Architecture/System Design

- Diagrams



Web App Architecture



Linux Architecture

Diagram tools

- Lucidchart
- Excalidraw
- draw.io

[Optional resource: draw.io / diagrams.net](#)

Architecture/System Design

- Internal/External APIs and Framework details

External API

Goal:

Obtain external data or services

Document:

- Endpoint(s) used
- Authentication
- Important limits / dependencies

Internal Backend API

Goal:

Support application features

Document:

- Route / endpoint
- Request type
- Input / output
- Error handling

Cloud / Hosting Service

Goal:

Run or store parts of the application

Document:

- Service used
- What it hosts/stores
- Cost / access / dependencies

Architecture/System Design

- Main Algorithms of System

Main Algorithm / Model

Goal:
What problem does it solve?

Inputs:
What data does it use?

Method:
What algorithm, model, or process is used?

Evaluation:
How will you know it works?

[Name] Algorithm / Model

Goal:
[Purpose in the system]

Inputs:
[Required data]

Method:
[How it works]

Evaluation:
[Metrics, tests, or success criteria]

How Are Sprints Graded?

- GitHub Project Board = source of truth.
- We look at the sprint commitment and whether tickets are meaningful and current.
- We look for completed, demo-able work and reasonable progress on unfinished work.
- We look at weekly updates, individual contributions, commits / pull requests, and code merged to main.
- The goal is steady, substantive progress — not activity for activity's sake.

Optional resource: Course syllabus — Sprint Progress

Simple Sprint Rubric — Student Version

Points	Category	What we look for
25	Sprint planning & board	Meaningful tickets, clear owners, reasonable commitment, board kept current
25	Technical progress	Demo-able functionality, merged code, tests or evidence of working progress
25	Individual contribution	Visible work through tickets, updates, commits / PRs, merged code, or other substantive contribution
25	Sprint process & communication	Weekly updates, blockers identified, plans refined, teamwork visible

Commit count alone is not a grading metric; substance and steady progress matter.

What Counts as Good Evidence?

- Working feature or working component
- Backend endpoint, database / data pipeline, model, algorithm, or integration work
- Meaningful completed tickets and clear progress on unfinished work
- Tests, experiments, bug fixes, or deployment steps
- Pull requests and code merged to main
- Basic UI tied to a real user story
- Clear weekly updates that make individual progress visible



What Does Not Help Much by Itself?

- Fancy visual styling without new functionality
- Many tiny or artificial commits
- Tickets with no clear deliverable or acceptance criteria
- A large unmerged branch at the end of the sprint
- Vague weekly status updates
- Marking work complete when the acceptance criteria are not complete

Commit count alone is not the grade. Quality, substance, and steady progress matter more.

How Sprint Grading Works

- TA performs detailed sprint grading.
- Professors do high-level project check-ins during class.
- The same rubric and expectations apply across teams.
- TA reviews: Project Board, weekly updates, tickets, commits / pull requests, merged code, and evidence of technical progress.
- Instructors may review unusual or unclear cases.

TA Support

- Tuesday and Wednesday, 3:30–5:30 PM
- Available by appointment
- Send the TA a message on Slack to schedule
- Good for: GitHub, sprint questions, project development, and technical questions

[Optional resource: GW GitHub information](#)

Product Requirements Document: Why?

- Writing assignments build toward a coherent design / requirements document for your Senior Design project.
- The goal is to connect the project summary, technical design, product specifications, constraints, and open questions.

Product Requirements Document

Section 1: Proposal

Section 2: Specs

Section 3: Considerations, Constraints, Dependencies

Section 4: Open Questions

Product Requirements Document

- Section 1: Proposal
 - Project Summary
- Section 2: Specs
 - Technical Summary
 - Product Specifications: user stories, basic mockups, flow diagrams
 - Technical Specifications: languages, architecture, frameworks, APIs
- Section 3: Considerations, Constraints, Dependencies
- Section 4: Open Questions
- See the course website for current assignment requirements and deadlines.

Optional resource: Course website

Presentation 1: Elevator Pitch

- Goal
- Convince a nontechnical audience that what you are building is a strong idea and that you have a realistic approach.
- Build from your project summary / proposal.
- Audience: investors, upper management, or another nontechnical stakeholder.
- Current format
- 4 minutes presentation + 2 minutes Q&A
- What are you building and why? Who are the users? What are the goals? How is it different from current products or research?
- Be ready for nontechnical questions.
- See the course website for current logistics and rubric.

Optional resource: Presentation requirements

Writing 2: Technical Summary

- Goal
- Explain why your technical approach is feasible and how you plan to move forward.
- Focus on:
 - • the technical component you are responsible for
 - • technical challenges / innovation
 - • tools, technologies, and dependencies
 - • key objectives and open questions
 - • feasibility, risks, and mitigation
 - • milestones and next steps
- See the course website / Blackboard for current submission requirements.

Optional resource: Course website

For Next Week

- Weekly Focus
- Continue technical development; prioritize working functionality.
- GitHub / Sprint
- Keep the Project Board current.
- Post your weekly status update.
- Make individual contributions visible.
- Use PRs and merge working code to main when appropriate.
- Mentor / Support
- Continue mentor meetings.
- Ask for help early on Slack or during TA support time.
- Functionality first — basic UI is enough when it supports the user story.

Optional Resources

- [Apple Human Interface Guidelines — UI patterns and platform conventions](#)
- [Material Design — reusable UI components and interaction guidance](#)
- [IBM Design Language — design-system examples](#)
- [Figma — wireframes and lightweight prototypes](#)
- [GW Tools & Services — Canva, Adobe, GitHub, and other student tools](#)
- [Lucidchart — diagrams and flows](#)
- [Excalidraw — quick collaborative sketches](#)
- [draw.io / diagrams.net — architecture and flow diagrams](#)
- [GitHub Docs — pull requests, Projects, Actions, and APIs](#)
- [Course website — current schedule, assignments, rubrics, and links](#)